

Version Control

26-09-2025

Adimalla Naga Manohar

1) What is version control?

Version control, also known as revision control or source control, is a practice and a set of tools that track changes to files. It allows you to save snapshots, compare versions, go back to earlier states, and collaborate without overwriting each other's work.

Key concepts (important terms you should know):

Repository (repo), the database storing all tracked files and their history (commits). This can be local or remote.

Working directory / Working tree, the files on disk you're editing.

Index / Staging area, a buffer where you prepare exactly what will be part of the next commit.

Commit, an immutable snapshot of the repository with metadata (author, message, timestamp) referencing tree objects (the file contents).

Branch, a movable pointer to a commit; this is used to isolate different lines of development.

HEAD, a pointer to the current commit or branch reference.

Tag, a named pointer, often used for releases.

Merge / Rebase, operations to integrate changes across branches.

Remote, a named link to another repo, such as origin on GitHub or GitLab.

Hash, a content-addressed identifier (SHA-1/SHA-256) that uniquely identifies commits or objects as used in Git.

Why a version control system (VCS) is more than just a backup: it keeps a record of changes (history and diffs) and metadata (author, reason). This supports organized collaboration, traceability, and reproducibility.

2) Why is version control important? (especially in automation)

Automation projects include more than just software. They also cover PLC code, HMI/SCADA projects, robot motion scripts, CAD or electrical drawings, test artifacts, procedures, calibration data, and deployment scripts. Version control provides order and safety.

Main reasons and benefits:

Traceability and auditability, showing who changed what, when, and why. This is crucial for regulated industries like pharmaceuticals, food, and automotive.

Rollback and recovery allow you to quickly revert to a known good configuration if a new change breaks production or causes unsafe behavior.

Collaboration and parallel work enable multiple engineers to work on different features or areas without interfering with each other.

Branching for isolation helps test risky changes, like a new PLC sequence or updated robot trajectory, on branches before merging them into production.

Reproducibility ensures you can recreate a build or deployed state from a specific commit or tag.

Change review and quality gates, through pull requests, merge requests, and code reviews, help reduce bugs and unsafe releases.

Disaster recovery involves storing controller or project backups in a repo or an asset management system.

Documentation and knowledge transfer occur as commits, messages, and tags create a narrative useful for new engineers.

Integration with CI/CD includes automated tests, static checks, and deployment pipelines, which are useful for simulating tests of control logic.

Configuration management ensures consistent deployments across multiple controllers or lines.

Notes specific to automation:

PLC binaries and vendor project files are often binary, making diffs difficult. Best practices include exporting human-readable formats where possible (like structured text or XML), utilizing vendor-provided export tools, or using Git LFS for large binary files.

For safety-critical systems, keep separate validated or stable release branches, and implement strict access controls and protected branches to avoid accidental pushes.

Use tags for deployed configurations, following semantic versioning (e.g., prod-v2.3.1), and maintain change logs for each release.

3) Different types of version control — explained

There are three main models of version control systems:

A. Local Version Control

Early systems were local-only tools that kept historical versions on the local file system, such as RCS.

How it works: snapshots are stored locally with no built-in sharing or networking.

Use-case: suited for single developers and quick local experiments.

Limitations include no collaboration, no central authority, and the risk of losing history if the machine fails.

B. Centralized Version Control (CVCS)

Examples are Subversion (SVN), CVS, and Perforce, which can be centralized.

Model: a single central server stores the repository. Clients check out working copies and commit back to the server.

Pros include a simple mental model, centralized access control and backups, and ease of learning.

Cons involve a single point of failure (if the server is down, everyone is affected), historically clunkier branching and merging compared to Git (though SVN has improved), and reliance on a network for many operations (you can't commit offline).

Automation relevance: some industrial shops still use this method for its straightforward central backup and admin control, which managers prefer.

C. Distributed Version Control (DVCS)

Examples include Git and Mercurial (hg).

Model: every developer has a complete copy of the repository, including history and commits. You push and pull between repos.

Pros include full local operations (commits, branches, history) that are fast, powerful branching and merging workflows, and no single point of failure.

This model is better suited for branching strategies and feature isolation.

Cons include a slightly steeper learning curve conceptually and the need for governance regarding push rights, naming, and branch policies.

Automation relevance: Git is the dominant choice in modern development and is highly recommended for automation teams due to its flexible branching, integration with CI/CD, and offline commit capabilities.

Hybrid / Enterprise solutions

There are specialized asset and version managers for automation, such as Rockwell FactoryTalk AssetCentre and Siemens Teamcenter. These tools combine repository-like features with targeted deployment, roles, and auditing. They are often used in highly regulated production environments to manage binary deployments, access control, and history.

4) Important Git commands, explained, with examples and best practices

Below I'll group commands by theme and explain what they do, why you use them, and safe patterns (including how to undo mistakes). I'll also show internal concepts where helpful.

Quick note on Git internals (conceptual): Git stores objects, including blobs for file content, trees for directories, and commits that reference trees. Every commit points to a tree and a parent commit or commits. Branches are simply named references pointing to commits. SHA hashes uniquely identify commits. Understanding this helps with recovery (for example, git reflog).

Setup & basic repo lifecycle

```
# configure identity (do this once per machine)
git config --global user.name "Alice Engineer"
git config --global user.email "alice@example.com"
```

initialize a new repo

git init # creates .git and makes current folder a repo

clone an existing remote repo

git clone git@github.com:org/project.git

git init creates a new repo. Use git clone to pull an existing repo, which is a copy of a remote.

git config controls user identity and other settings.

Inspecting state

git status # shows modified, staged, and untracked files

git diff # shows changes in the working tree that are not staged

git diff --staged # shows staged changes compared to the last commit

git status is arguably the most used command. It tells you what Git thinks about your working tree.

Use git diff before staging or committing to see what changes you made.

Staging & committing

git add file1.txt # stage a file

git add -p # interactively choose parts to stage (highly recommended)

git commit -m "Short summary" -m "Longer description and rationale" # create a commit

git commit -S -m "Signed commit" # sign commits (GPG)

Commit messages should have a short first line (up to 50 characters), followed by a blank line, and then details. Include issue or ticket references.

Use git add -p to make atomic, focused commits by staging chunks or parts.

-S signs commits for non-repudiation, which is useful in environments requiring high integrity.

Branching & switching

git branch # list branches

git branch feature/x # create a branch

git checkout feature/x # switch (older method)

git switch -c feature/x # modern: create and switch

git switch feature/x # switch

Branches are cheap and essential for isolation. Create a branch for each feature, bug fix, or risk.

Use switch instead of checkout when possible for clarity.

Merging vs Rebasing (and when to use each)

Merge (keeps history graph):

```
git checkout main
git merge feature/x
```

Rebase (linearizes history, rewrites commits):

```
git checkout feature/x
git rebase main
# then switch to main and fast-forward or merge
```

Merge preserves the true history and is safer in shared branches by generating a merge commit.

Rebase replays commits on top of another branch. It makes history linear and clean but rewrites commit hashes, so do not rebase public or shared branches. Only rebase local feature branches.

In teams, prefer rebase for local cleanup and use merge or squash-and-merge when integrating into main, depending on your policy.

Working with remotes

```
git remote -v          # list remotes
git remote add origin <url>  # add a remote
git fetch origin        # fetch remote references (no merge)
git pull               # fetch and merge (default)
git pull --rebase       # fetch and rebase (preferred by some people)
git push origin feature/x
git push --set-upstream origin feature/x # set local branch to track remote
```

Fetch is safer than pull because it doesn't change local branches.

Push sends commits to the remote. Use --set-upstream the first time for convenience.

Undoing things — safe and dangerous commands

Revert (safe, non-destructive)

```
git revert <commit-hash> # creates a new commit that undoes changes
```

Use revert to undo changes in published branches as it adds a new commit.

Reset (potentially destructive locally)

```
git reset --soft <commit> # move HEAD and keep staged and working changes
git reset --mixed <commit> # default: move HEAD; keep working changes but unstage them
git reset --hard <commit> # move HEAD; reset the index and working tree (DANGEROUS)
```

--hard destroys local changes, so use it with care.

git reflog can recover references in many cases (even after resets), but this is not always simple.

Checkout/restore for files

```
git restore file.txt          # discard working tree changes (newer command)
git checkout -- file.txt      # older method to restore a file from HEAD
git restore --staged file.txt # unstage a file
```

Stash (temporary storage)

```
git stash          # save working tree and index
git stash list
git stash apply    # reapply changes (leave stash)
git stash pop      # reapply and drop from stash
```

Use stash when you must switch branches but have uncommitted work.

Force push — use with caution

```
git push --force-with-lease origin feature/x
```

--force-with-lease is safer than regular --force because it only force pushes if the remote hasn't changed unexpectedly.

Never force push to shared protected branches, like main, unless you coordinate and understand the consequences.

Inspecting history & finding breaks

```
git log --oneline --graph --decorate --all
git show <commit-hash>      # show commit details and diff
git blame file.txt          # find out who last changed each line
git bisect start
git bisect bad <bad-commit>
git bisect good <good-commit>
# then run tests until bisect finds the offending commit
```

git bisect is invaluable for finding the commit that introduced a regression. It performs a binary search between good and bad commits.

git blame helps identify the author or responsible change for a line.

Tagging & releases

```
git tag v1.2.3              # lightweight tag referencing the current commit
git tag -a v1.2.3 -m "Release" # annotated tag with a message
git push origin v1.2.3      # push tags explicitly
git push --tags             # push all tags
```

Use annotated tags for releases because they store author information, date, and message. They can also be signed.

Advanced & collaboration commands

```
git cherry-pick <commit>    # apply a single commit from another branch
git merge --no-ff feature/x  # create a merge commit even if it could have fast-forwarded
git rebase -i main          # interactive rebase to reorder, squash, or edit commits
git submodule add <repo> path # include another repository as a submodule
git submodule update --init --recursive
```

Cherry-pick is useful for selectively applying fixes between branches, but use it sparingly.

Rebase -i is great for cleaning up a messy local branch by reordering, squashing small commits, or editing messages.

Submodules let you include external repositories but can be tricky, so use them with care. Alternatives include subtrees, monorepos, and package registries.

Git LFS (Large File Storage)

For binary assets common in automation, like CAD or PLC binaries, use Git LFS to store large files outside normal object storage to keep the repo fast.

Install LFS and track patterns:

```
git lfs install
git lfs track "*.dwg"
git add .gitattributes
```

LFS stores pointers in Git, while the actual content is in separate LFS storage.

References:-

Pro Git Book (Official)

Scott Chacon and Ben Straub. *Pro Git* (2nd Edition). Apress.

Available free online at: <https://git-scm.com/book/en/v2>

Git Documentation (Official Manual)

Git — Documentation.

Available at: <https://git-scm.com/doc>

Atlassian Git Tutorials

Atlassian. “*What is Git?*” and related tutorials.

Available at: <https://www.atlassian.com/git/tutorials/what-is-git>

Automation World Article

“*Make Version Management a Best Practice*”, Automation World.

Available at:

<https://www.automationworld.com/products/control/article/13300798/make-version-management-a-best-practice>

Wikipedia Overview

“*Version Control*” — Wikipedia contributors.

Available at: https://en.wikipedia.org/wiki/Version_control